

Penetration Testing & Application



PENETRATION TESTING REPORT

Target: UHIS Mini Portal (“Horus Byte”)

Type: Custom Web Application | Platform: Docker (Local Training Environment) | Stack: Node.js / Express / MySQL

Prepared By: **Ahmed Abdelmoneim Kotb Abdelmoneim**

Date of Issue: **26 August 2026**

Classification: **Internal – Training Use Only**

Table of Contents

Document Control	1
Document Details	1
Document History.....	1
Summary of Findings	2
Project Scope	3
Project Time Line	3
Project Team.....	3
1. Executive Summary.....	4
2. Methodology.....	4
3. Tools Used	5
4. Reconnaissance	5
5. Vulnerability Findings	7
6. Proof of Concept — Exploitation Walkthrough	10
Step 1 — Session Setup & Baseline Recon	10
Step 2 — Exploiting SQL Injection.....	11
Step 3 — Exploiting Broken Access Control (IDOR)	12
Step 4 — Exploiting Stored XSS.....	13
Step 5 — Authentication & Session Weaknesses	14
Step 6 — Remediation Verification.....	15
7. Confirmed Impact Summary.....	17
8. Appendix A — Additional Testing Evidence (WSTG-CONF / WSTG-ATHN).....	17
9. Appendix B — Remediation Verification Evidence	20
10. Recommendations Summary.....	23
11. Conclusion	23

Document Control

Document Details

Field	Details
Document Title	Horus Byte — UHIS Mini Portal Penetration Testing Report V1
Classification	Internal – Training Use Only
Description	This document includes the technical details of the penetration test performed against the UHIS Mini Portal (“Horus Byte”), a self-built training prototype simulating Egypt's Universal Health Insurance System, covering reconnaissance, exploitation, and remediation verification.
Target	UHIS Mini Portal — http://localhost:3000 (Dockerized local training environment)
Date of Issue	26/08/2026

Document History

Version	Date	Author	Summary of Changes
1.0	26/08/2026	Ahmed Abdelmoneim Kotb	Initial penetration testing findings, exploitation walkthrough, and remediation verification

Summary of Findings

The figures and table below elaborate on the summary of the penetration testing results.

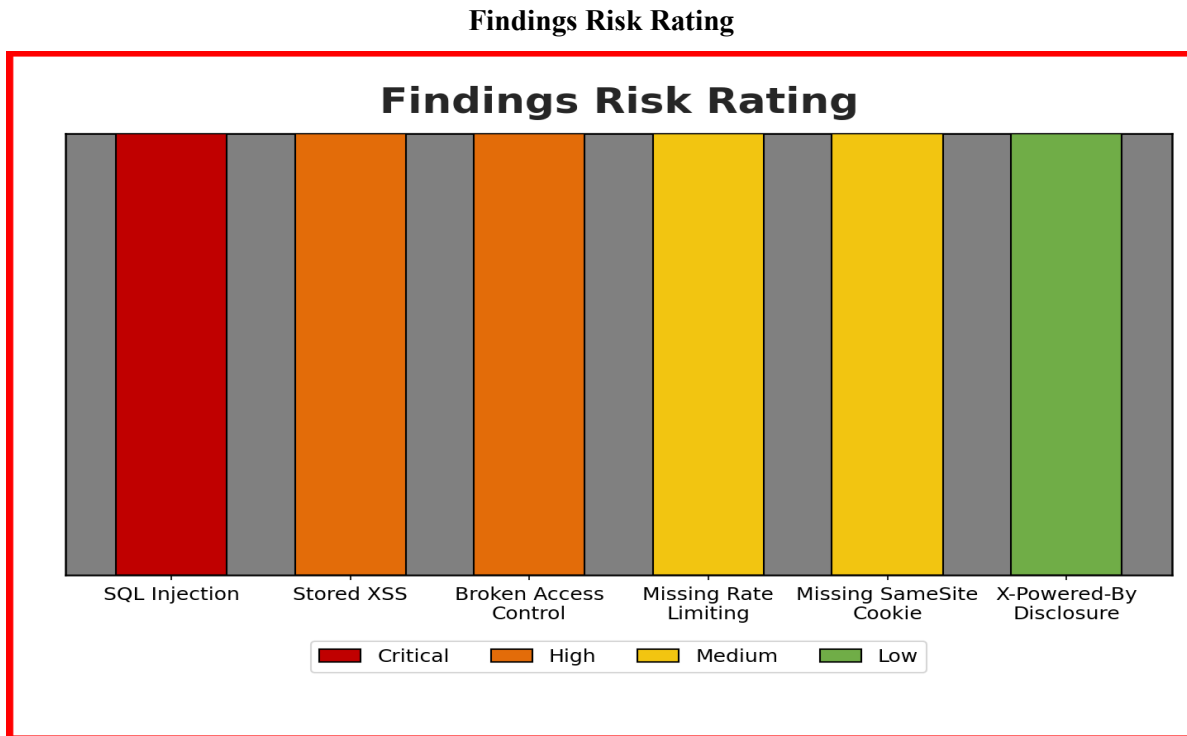


Figure A — Risk rating for each of the six findings identified during the assessment.

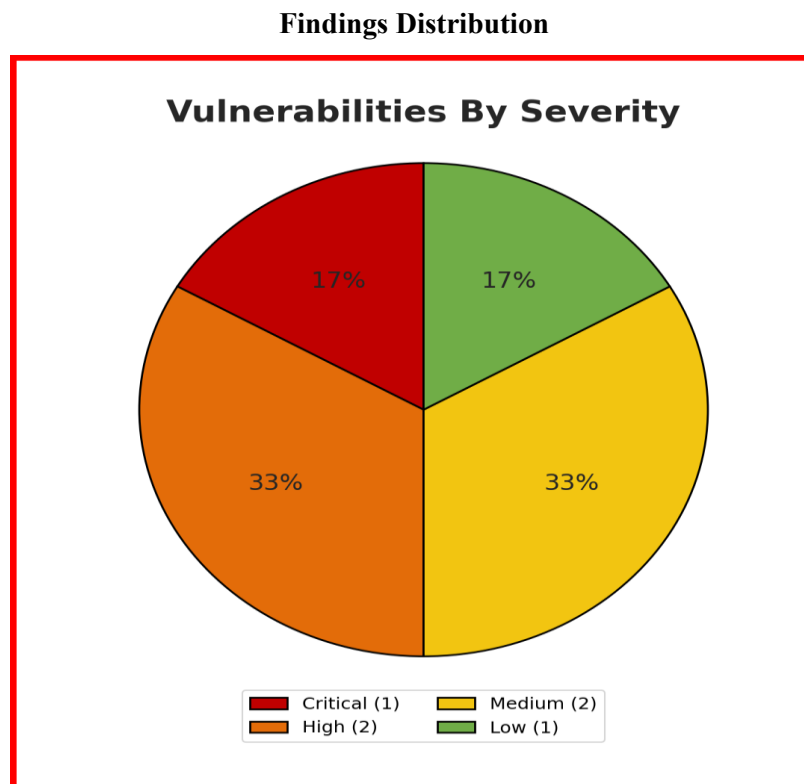


Figure B — Distribution of findings by severity across the engagement.

The table below summarizes the vulnerabilities identified during the assessment, ordered by discovery sequence.

#	Vulnerability	CVSS 3.1	Risk
1	SQL Injection (UNION-Based Data Exfiltration)	9.1	Critical
2	Stored Cross-Site Scripting (XSS)	8.2	High
3	Broken Access Control (IDOR)	7.5	High
4	Missing Rate Limiting on Login	5.3	Medium
5	Missing SameSite Cookie Attribute	4.3	Medium
6	X-Powered-By Information Disclosure	2.4	Low

Project Scope

Item	Value
Target Name	UHS Mini Portal (Horus Byte)
Target URL	http://localhost:3000
Stack	Node.js / Express / MySQL, Dockerized
Open Ports	3000 (HTTP application)
Test Type	Gray-box, authenticated assessment (valid low-privilege account provided)
Access Method	Local Docker environment, tested via Burp Suite Community Edition

Project Time Line

Item	Detail
Assessment Dates	22–26 August 2026
Duration	Multi-day engagement — reconnaissance through remediation verification
Environment	Local Docker-based training lab (isolated, no external network exposure)

Project Team

Names	Titles	Roles
Ahmed Abdelmoneim Kotb	Penetration Tester	Full engagement — reconnaissance, exploitation, remediation, and reporting

1. Executive Summary

This report documents a penetration test conducted against “Horus Byte,” a self-built training prototype simulating a portal for Egypt's Universal Health Insurance System (UHS). The application was developed and tested as part of the Penetration Testing & Application Security training program, using a gray-box, authenticated methodology consistent with the OWASP Web Security Testing Guide (WSTG) black-box principle of minimal prior knowledge.

The assessment identified six vulnerabilities spanning Critical to Low severity. The most significant finding was a SQL Injection vulnerability in the insurance case search endpoint that allowed complete exfiltration of the users table — including bcrypt password hashes for every citizen, doctor, and administrator account — via a single UNION-based payload. This was compounded by:

- A Stored Cross-Site Scripting (XSS) vulnerability in the medical notes feature, allowing persistent JavaScript execution and session cookie theft for any user who later viewed an affected patient file.
- A Broken Access Control (IDOR) flaw in the patient file endpoint, allowing any authenticated doctor to read medical records belonging to patients and doctors outside their assigned governorate, with no authorization check performed.
- Missing rate limiting on the login endpoint, permitting unlimited password-guessing attempts against any known account.
- A session cookie missing the SameSite attribute, weakening the application's built-in CSRF resistance.
- Disclosure of the underlying Express.js framework via the default X-Powered-By response header.

All six findings were successfully reproduced, exploited, and subsequently remediated in the application's source code. Each fix was independently re-verified using the same reproduction steps used during initial exploitation, confirming the vulnerabilities no longer exist. Detailed findings, proof-of-concept evidence, and remediation guidance are provided in the following sections.

2. Methodology

The assessment followed the OWASP Web Security Testing Guide (WSTG) black-box methodology — testing with minimal prior knowledge of the application's internals — structured across the following phases:

- WSTG-INFO — Information Gathering: mapping every endpoint via Burp Suite's proxy history and reviewing response headers for technology disclosure.
- WSTG-CONF — Configuration & Deployment: probing for exposed files (.env, package.json), checking allowed HTTP methods and directory listing behavior.
- WSTG-ATHN — Authentication Testing: assessing error-message consistency (user enumeration) and brute-force / rate-limit behavior on the login endpoint.
- WSTG-SESS — Session Management: inspecting session cookie flags (HttpOnly, Secure, SameSite) and session invalidation behavior on logout.
- WSTG-ATHZ — Authorization Testing: comparing a doctor's legitimate patient list against files actually reachable by ID manipulation.
- WSTG-INPV — Input Validation Testing: testing every input field with a single quote and a harmless HTML tag before attempting a full payload, to confirm the absence of sanitization.
- Exploitation & Verification — confirming and deepening each finding using Burp Suite Repeater.
- Remediation & Re-Testing — applying source-code fixes and re-running the exact same reproduction steps to confirm each vulnerability was closed.

3. Tools Used

The following tools were used throughout the reconnaissance, exploitation, and remediation-verification phases of this assessment.

Tool	Category	Purpose
Burp Suite Community Edition	Web Proxy / Interception	Intercepting, replaying, and modifying HTTP requests via Proxy and Repeater
Google Chrome	Web Browser	Interacting with and manually testing the UHIS Mini Portal application
Docker Desktop	Containerization Platform	Running the target application and its MySQL database locally
Node.js / npm	Runtime	Running and rebuilding the application after remediation fixes
MySQL (via mysql2 driver)	Database	Backing datastore targeted by the SQL Injection testing

4. Reconnaissance

Reconnaissance began by logging in with a provided low-privilege doctor account (Dr. Ahmed Kotb) and browsing the application normally with Burp Suite's proxy running, to map every page, form, and API call before any testing began.

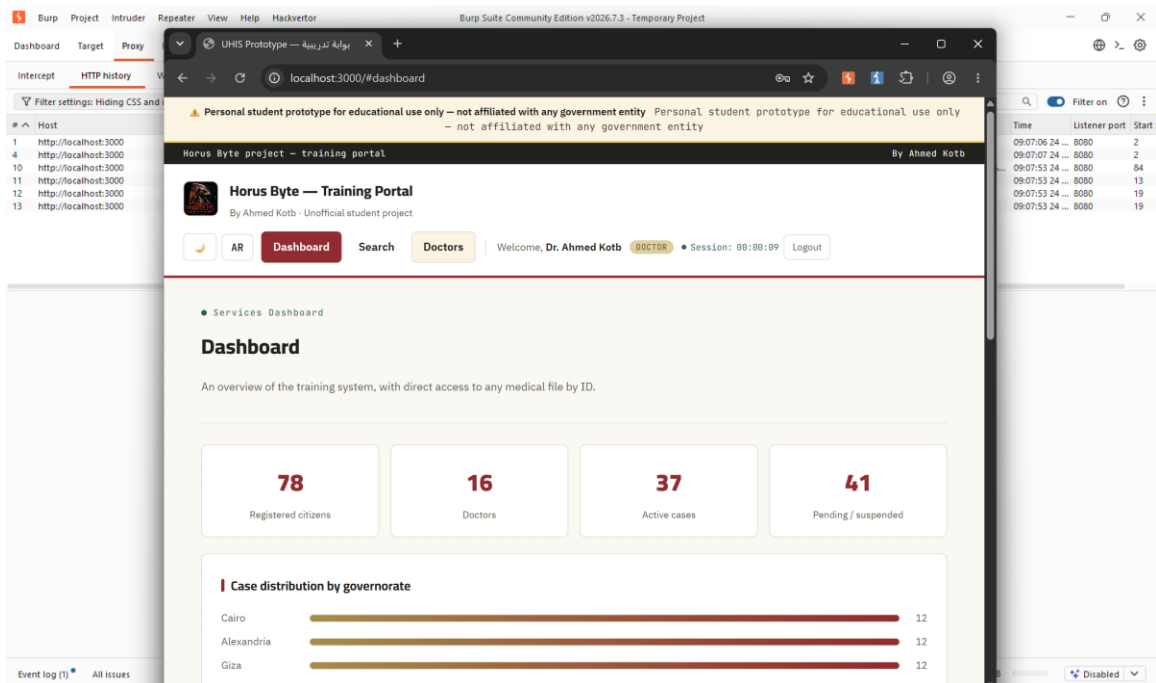
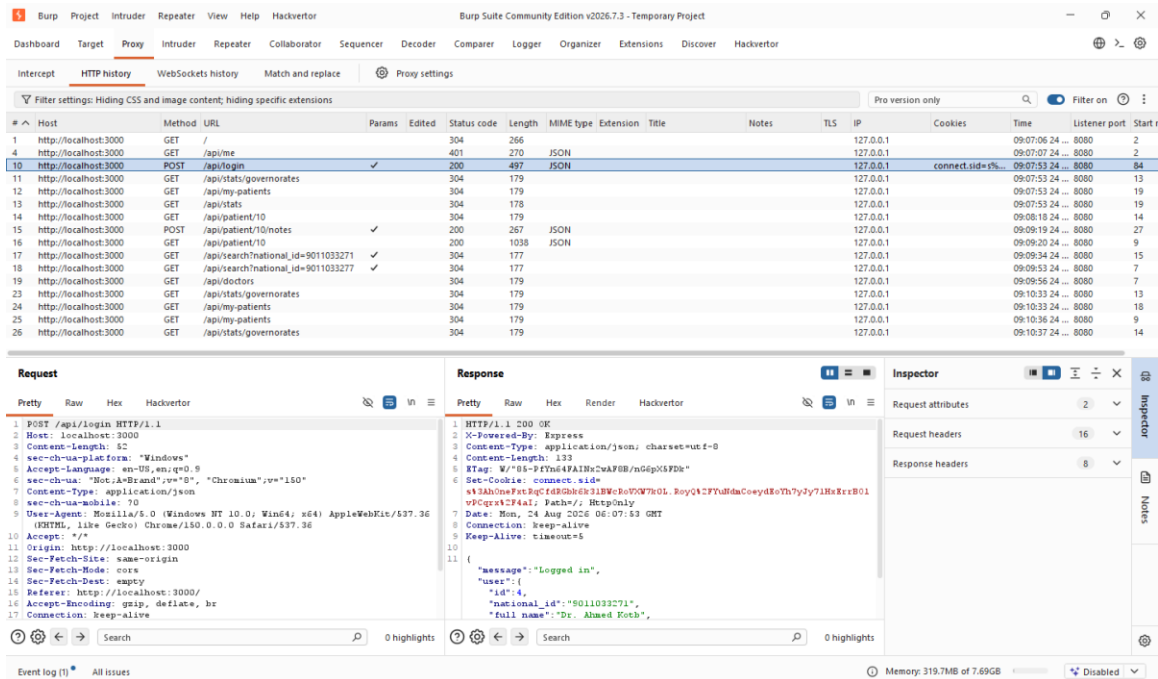


Figure 1 — The Horus Byte dashboard after login, showing system-wide statistics and the governorate case-distribution widget.

The following endpoints were identified and catalogued via Burp Suite's HTTP history before any testing began:

Method	Endpoint	Purpose
POST	/api/login	Authenticate and establish a session
GET	/api/me	Return the current session's user
GET	/api/search?national_id=	Look up an insurance case by national ID
GET	/api/patient/:id	Retrieve a patient's medical file and notes
POST	/api/patient/:id/notes	Add a medical note to a patient file
GET	/api/my-patients	List the logged-in doctor's own assigned patients
GET	/api/doctors	Public directory of registered doctors
GET	/api/stats	Dashboard summary statistics
GET	/api/stats/governorates	Case counts grouped by governorate



The screenshot shows the Burp Suite interface. The top menu bar includes: Burp, Project, Intruder, Repeater, View, Help, Hackvector. The main toolbar shows: Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, Discover, Hackvector. The 'Proxy' tab is active, showing the 'HTTP history' section. A table lists HTTP history entries with columns: #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, Notes, TLS, IP, Cookies, Time, Listener port, and Start. The table shows various GET and POST requests to localhost:3000. The 'Request' and 'Response' sections are expanded for the first entry (POST /api/login). The 'Request' section shows the raw HTTP request details. The 'Response' section shows the raw HTTP response details, including the status code 200 OK and the response body in JSON format: {"message": "Logged in", "user": {"id": 4, "national_id": "9011033271", "full name": "Dr. Ahmed Kotb"}. The 'Inspector' section on the right shows the request and response headers.

Figure 2 — Burp Suite HTTP history showing the mapped endpoints, alongside the login response revealing session cookie details.

5. Vulnerability Findings

The purpose of this section is to go through each identified vulnerability, describe its impact, highlight the assets affected, and recommend a course of action to mitigate it.

#	Vulnerability Name	Severity (CVSS)	URL / IP
1	SQL Injection (UNION-Based Data Exfiltration)	Critical (9.1)	GET /api/search?national_id= — UHIS Mini Portal
Description: The national ID search endpoint concatenates user input directly into a raw SQL query without parameterization or input sanitization. This allows an attacker to alter the intended SQL logic. Using UNION SELECT, an attacker can pull data from arbitrary tables unrelated to the intended insurance_cases table — including the users table, which holds bcrypt password hashes for every citizen, doctor, and administrator account. Error-based and boolean-based injection were also confirmed, with the raw MySQL error message returned verbatim to the client.			
Impact: - Any logged-in user can extract the complete users table, including national IDs, roles, and bcrypt password hashes for every account, including administrators. - Boolean-based injection (' OR '1'=1) bypasses the intended single-record filter, returning all 78 insurance case records from a single request. - Leaked password hashes can be cracked offline to obtain valid admin credentials and fully compromise the application.			
Solution: - Replace all string-concatenated SQL with parameterized queries so user input is always treated as data, never as executable SQL. - Do not return raw database error details to the client; log them server-side only. - Apply least-privilege database accounts so the application account cannot access columns it does not need.			
For more information, please refer to: https://owasp.org/www-community/attacks/SQL_Injection			

#	Vulnerability Name	Severity (CVSS)	URL / IP
2	Stored Cross-Site Scripting (XSS)	High (8.2)	POST /api/patient/:id/notes (note_text field)
Description: The medical notes field accepts arbitrary text from any doctor account and stores it without sanitization. The frontend previously rendered each note using string-concatenated innerHTML, meaning any HTML or JavaScript submitted in a note is parsed and executed by the browser of every user who later views that patient file.			
Impact: - A malicious or compromised doctor account can persist JavaScript that executes automatically for any other user who opens the same patient file. - Executed script runs in the victim's session context and can read their session cookie, enabling session hijacking. - Because the payload is stored rather than reflected, it persists indefinitely until removed.			
Solution: - Never build DOM content by concatenating raw user input into innerHTML; use textContent so text is always rendered literally. - Sanitize any input that genuinely requires HTML formatting using an allow-list based library. - Set a Content-Security-Policy header restricting inline script execution as defense-in-depth.			
For more information, please refer to: https://owasp.org/www-community/attacks/xss/			

#	Vulnerability Name	Severity (CVSS)	URL / IP
3	Broken Access Control (IDOR)	High (7.5)	GET /api/patient/:id
Description: The endpoint returns a full patient medical file to any authenticated user. It verifies only that a session exists, never that the requesting doctor is actually assigned to that file. Sequentially incrementing the numeric id parameter allowed access to files belonging to unrelated patients across different governorates and doctors.			
Impact: - Any authenticated doctor can enumerate and read every patient's medical file in the system by simply incrementing the file ID. - Sensitive diagnosis data for patients with no relationship to the requesting doctor is exposed, violating patient confidentiality. - The same missing-check pattern could allow a citizen account to view another citizen's private medical records.			
Solution: - Add a server-side authorization check comparing the session user against the file's assigned doctor or owning citizen. - Return a generic 403/404 for unauthorized attempts rather than leaking whether the record exists. - Apply the same object-level authorization pattern already used correctly elsewhere, such as the my-patients endpoint. For more information, please refer to: https://owasp.org/www-community/Broken_Access_Control			

#	Vulnerability Name	Severity (CVSS)	URL / IP
4	Missing Rate Limiting on Login	Medium (5.3)	POST /api/login
Description: The login endpoint accepted an unlimited number of authentication attempts from the same client with no delay, lockout, or CAPTCHA. Repeated incorrect password attempts against a known national ID completed instantly with no throttling response.			
Impact: - An attacker who has obtained or guessed a valid national ID can perform an unthrottled brute-force attack against that account's password. - A predictable national ID format significantly lowers the effort required to compromise any account, including admin accounts.			
Solution: - Apply a per-IP and/or per-account rate limit on the login endpoint (e.g., 5 attempts per 15 minutes). - Log and alert on repeated failed login attempts for early detection of active brute-force attempts. For more information, please refer to: https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks			

#	Vulnerability Name	Severity (CVSS)	URL / IP
5	Missing SameSite Cookie Attribute	Medium (4.3)	Session cookie (connect.sid), set at POST /api/login
Description: The session cookie was issued with the HttpOnly flag but without the SameSite attribute, and without the Secure flag. Without SameSite, the browser attaches the session cookie to cross-site requests, weakening the application's built-in CSRF resistance for state-changing endpoints.			
Impact: - A malicious third-party site could trigger a forged request to a state-changing endpoint with the victim's session cookie attached automatically. - Absence of the Secure flag also means the cookie could be transmitted over plain HTTP if HTTPS is not enforced end-to-end.			
Solution: - Set SameSite=Lax (or Strict where feasible) on the session cookie. - Set the Secure flag on the cookie in any environment served over HTTPS. For more information, please refer to: https://owasp.org/www-community/attacks/csrf			

#	Vulnerability Name	Severity (CVSS)	URL / IP
6	X-Powered-By Information Disclosure	Low (2.4)	All HTTP responses
Description: Every response from the application included the default Express.js header X-Powered-By: Express, disclosing the backend framework in use to any observer with no effort required.			
Impact: Framework fingerprinting narrows the search space for an attacker looking for framework-specific known vulnerabilities.			
Solution: Disable the header globally (e.g., app.disable('x-powered-by') in Express). For more information, please refer to: https://owasp.org/www-community/Improper_Error_Handling			

6. Proof of Concept — Exploitation Walkthrough

This section provides a step-by-step walkthrough, with supporting evidence, of how each finding was discovered and exploited using Burp Suite.

Step 1 — Session Setup & Baseline Recon

Logged in with the provided low-privilege doctor account (Dr. Ahmed Kotb) and mapped the application's endpoints via Burp Suite's proxy history before any testing began.

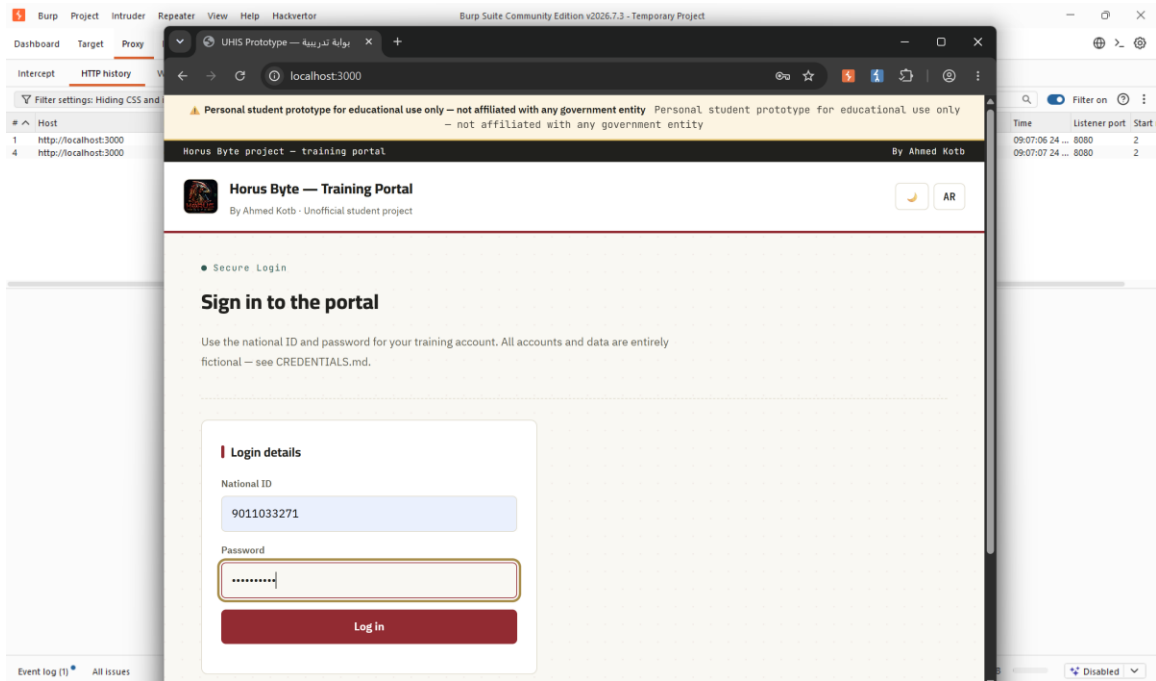


Figure 3 — The Horus Byte login screen.

The `national_id` parameter on the search endpoint was tested with a boolean payload (' OR '1'=1), which bypassed the intended single-record filter and returned all 78 insurance case records in one request.



A UNION-based payload targeting the users table was then used to exfiltrate every account's role, governorate, and bcrypt password hash — including administrator accounts.



Step 3 — Exploiting Broken Access Control (IDOR)

The legitimate patient list for the logged-in doctor was first retrieved via GET /api/my-patients, establishing which file IDs the account was actually authorized to view (files 1, 4, 7, and 10).

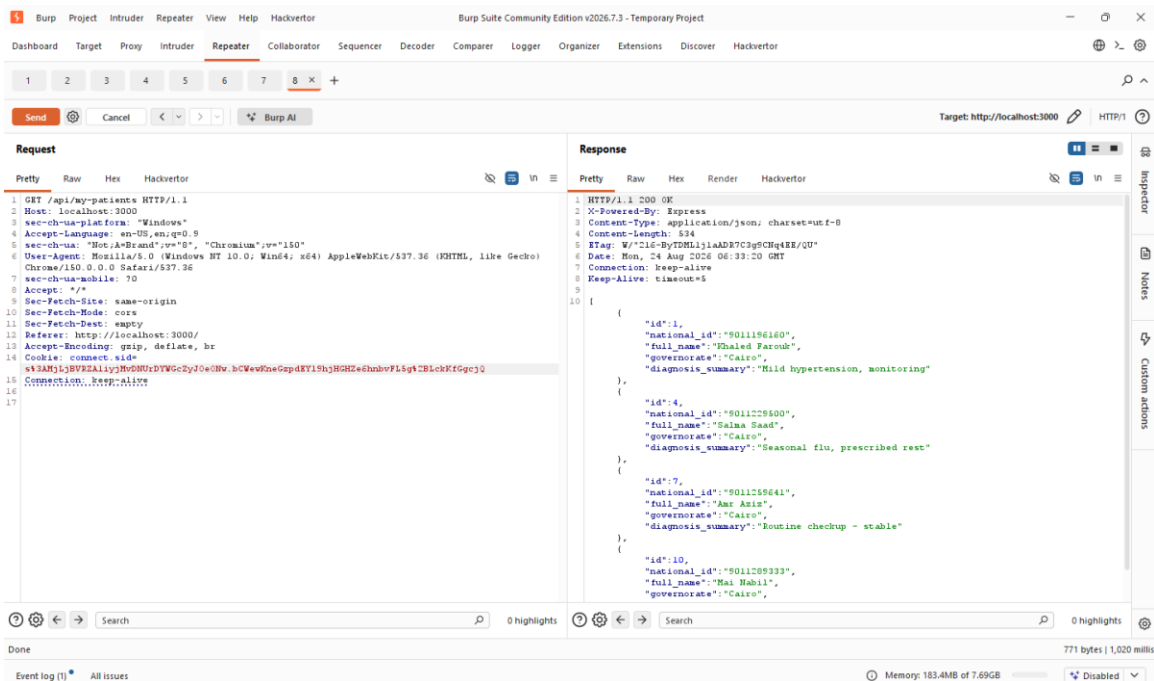


Figure 6 — The doctor's legitimate patient list, returned by /api/my-patients.

A file ID belonging to a different doctor (file 3) was then requested directly, with no ownership relationship to the logged-in account.

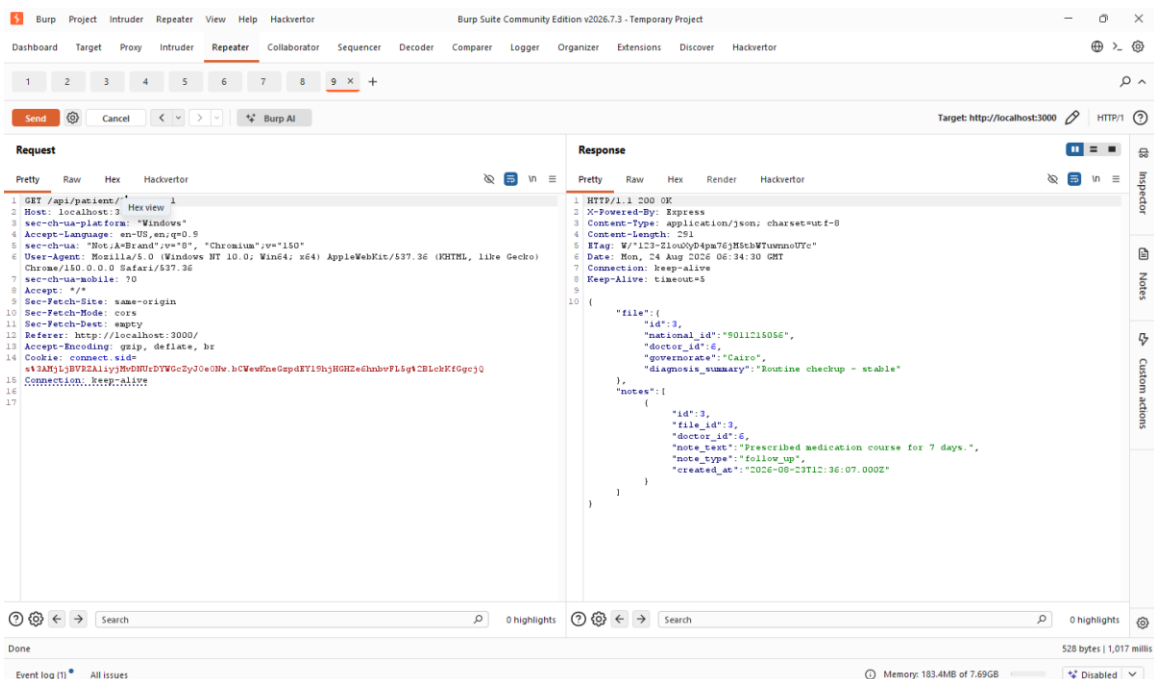


Figure 7 — Full medical file returned for a patient assigned to a different doctor, with no authorization check.

The same request pattern was repeated against a file belonging to a patient in a distant governorate (file 60, Ismailia), confirming the flaw was not limited to a single record.

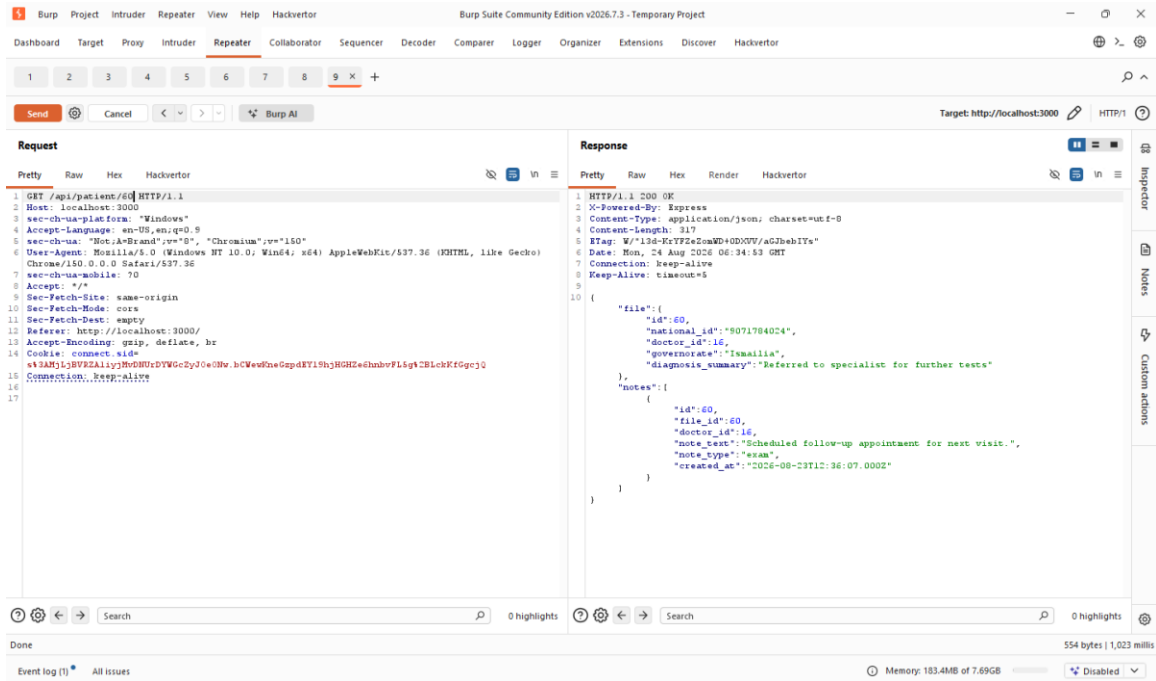


Figure 8 — Cross-governorate IDOR: a Cairo-based doctor account accessing a patient file in Ismailia.

Step 4 — Exploiting Stored XSS

A harmless HTML tag was first submitted via the “Add a new medical note” field to confirm the absence of input sanitization, followed by an event-handler-based payload (raw <script> tags are ignored by browsers when injected via innerHTML, but event handlers are not).

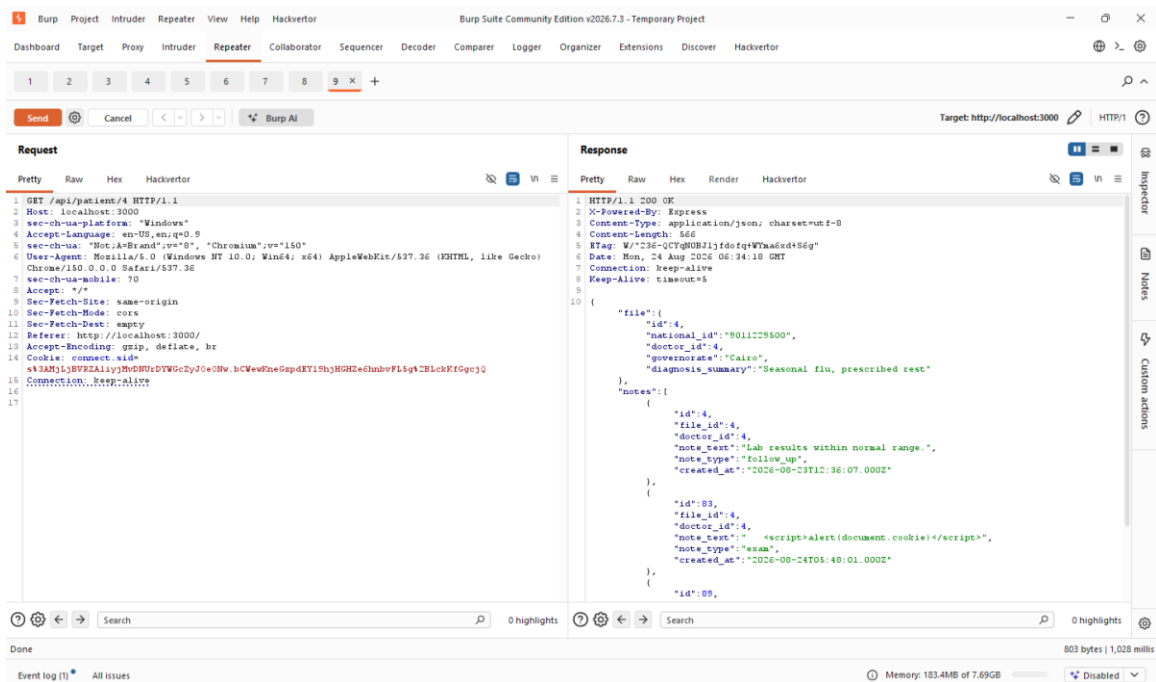


Figure 9 — Progression of test payloads stored and rendered in the medical notes list, including the final <script> payload.

Reloading the file confirmed the payload executed automatically, triggering a JavaScript alert() populated with the victim's session cookie — proof that session hijacking would be possible in a real attack.

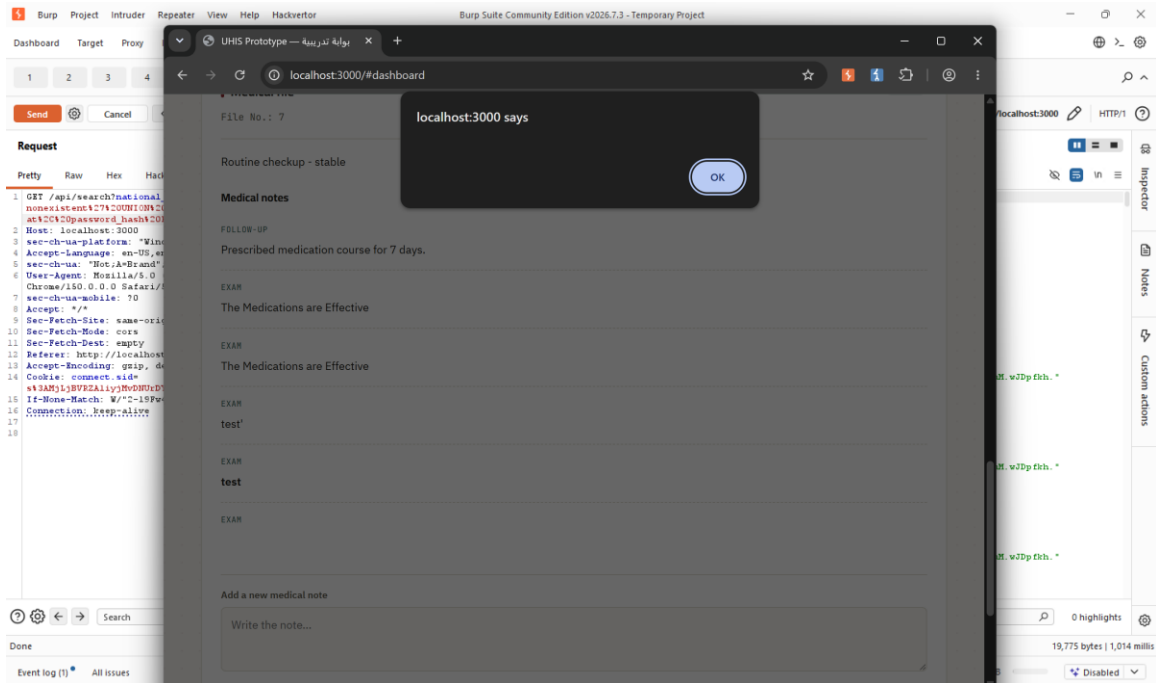


Figure 10 — JavaScript alert() executing automatically on page load, confirming exploitable Stored XSS.

Step 5 — Authentication & Session Weaknesses

The login endpoint was tested with an incorrect password and a non-existent national ID. Both returned an identical “Invalid credentials” message (a positive finding — no user enumeration), but repeated attempts revealed no rate limiting was in place at the time.

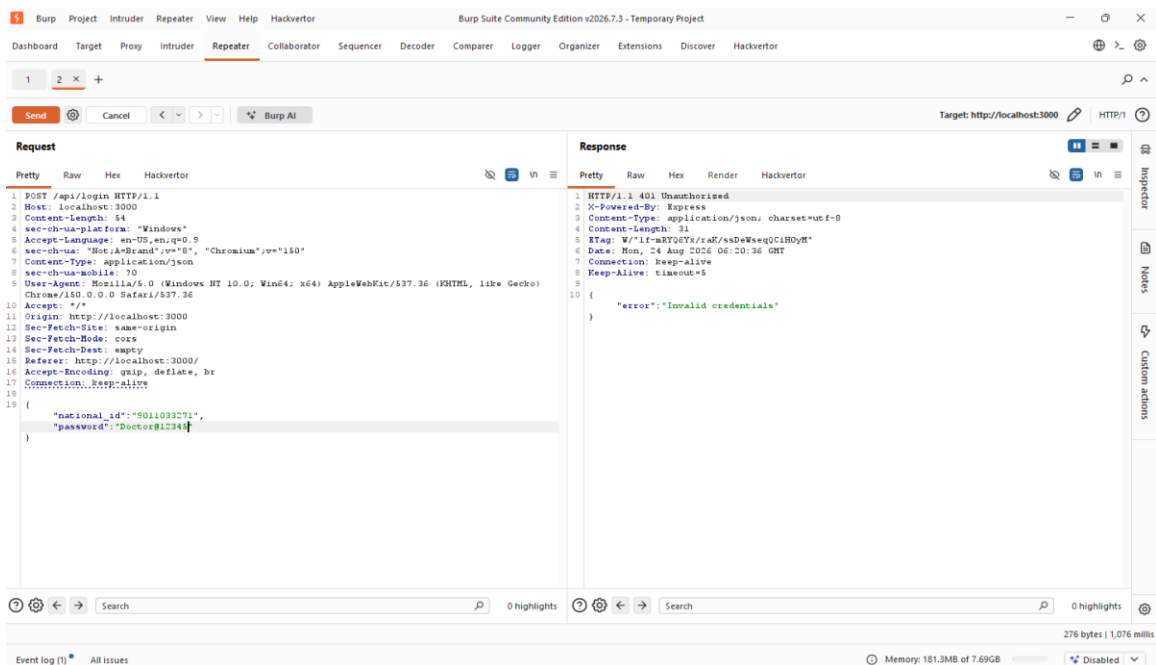


Figure 11 — Login response for an incorrect password, with no rate-limiting headers present.

The session cookie issued on successful login was inspected and found to include HttpOnly, but neither Secure nor SameSite were present.

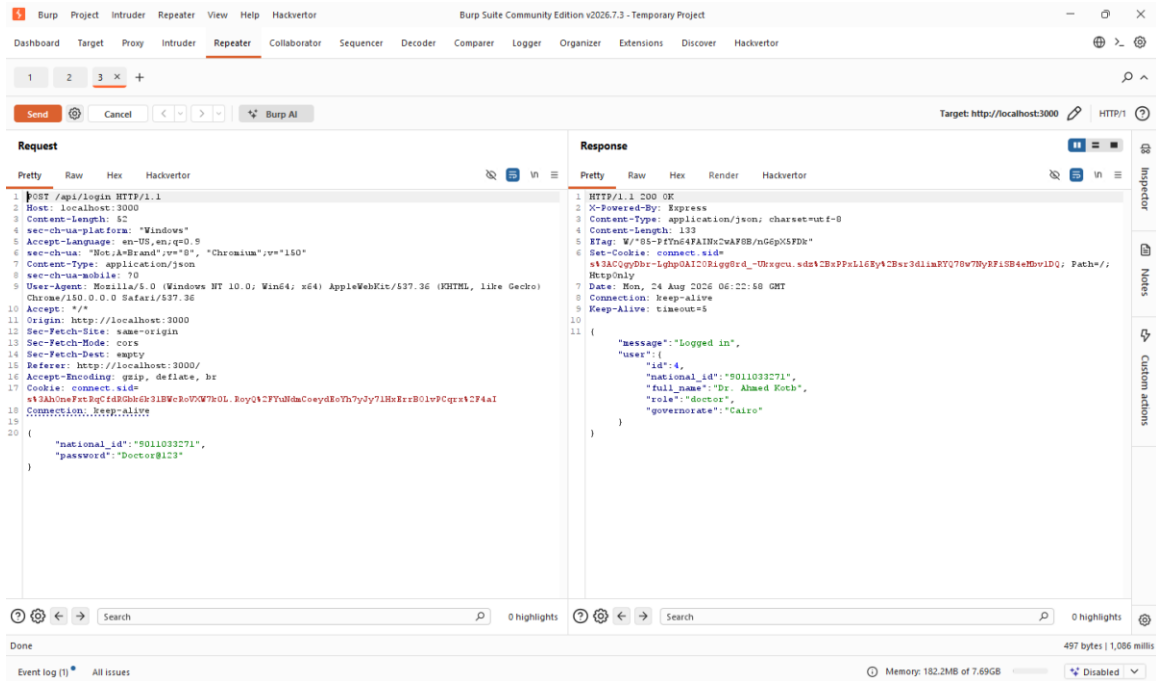


Figure 12 — Set-Cookie header on login, showing HttpOnly present but SameSite and Secure absent.

Step 6 — Remediation Verification

Following remediation, every exploitation step above was repeated exactly as performed originally, to confirm each vulnerability was closed without breaking normal application functionality.

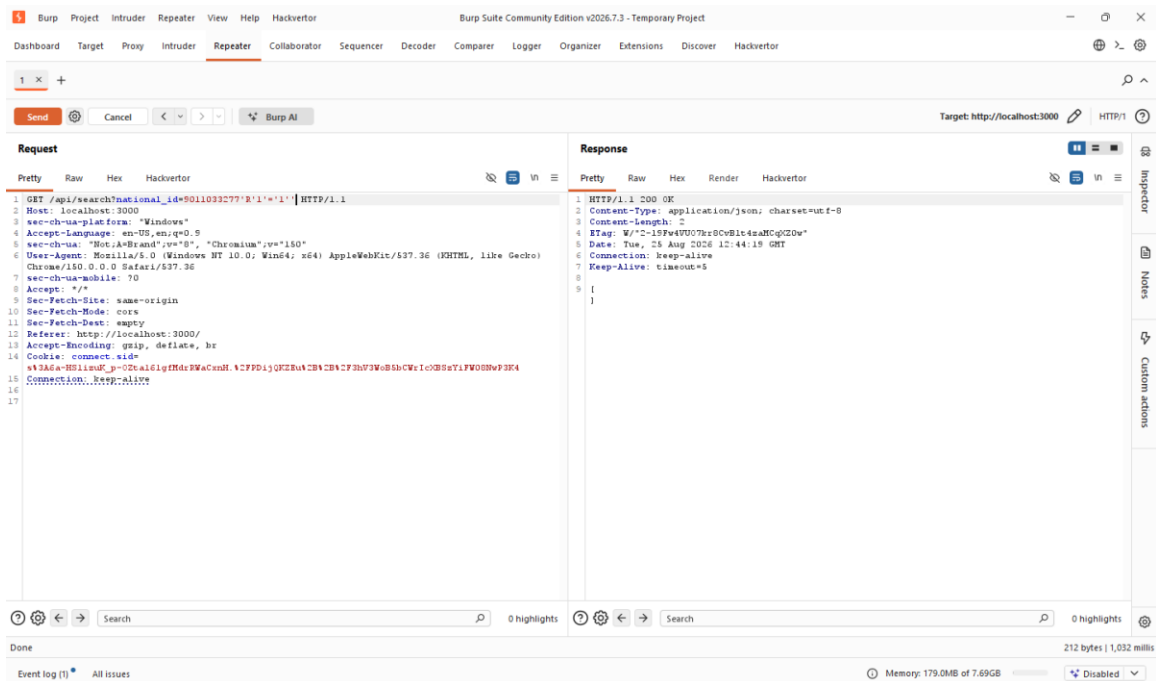


Figure 13 — After remediation: the same boolean-based SQLi payload now returns an empty result instead of leaking data.

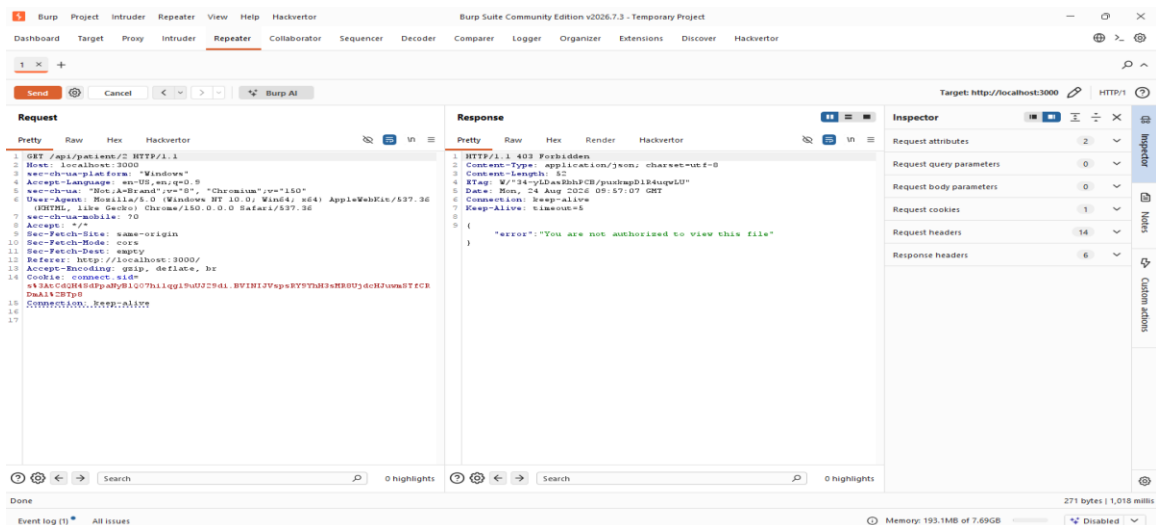


Figure 14 — After remediation: requesting an out-of-scope patient file now returns 403 Forbidden.

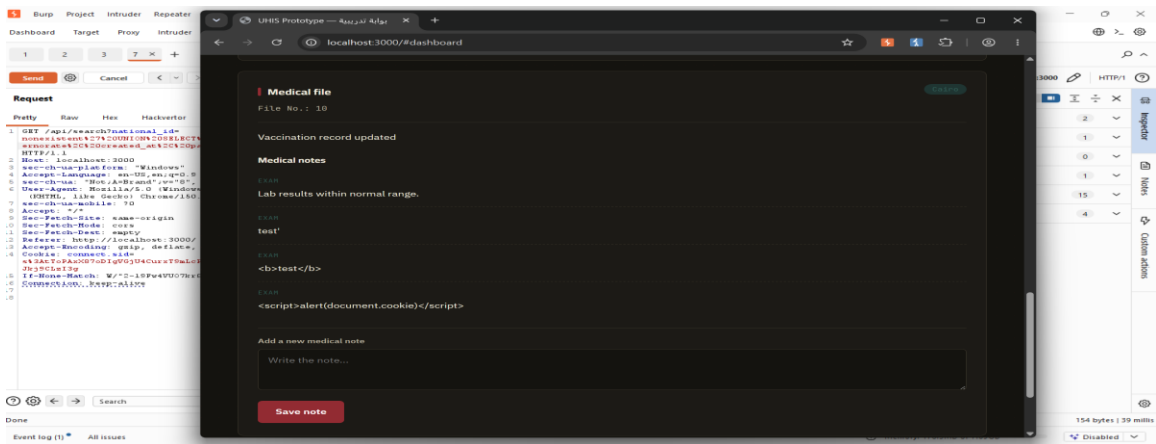


Figure 15 — After remediation: previously stored XSS payloads now render as literal, harmless text.

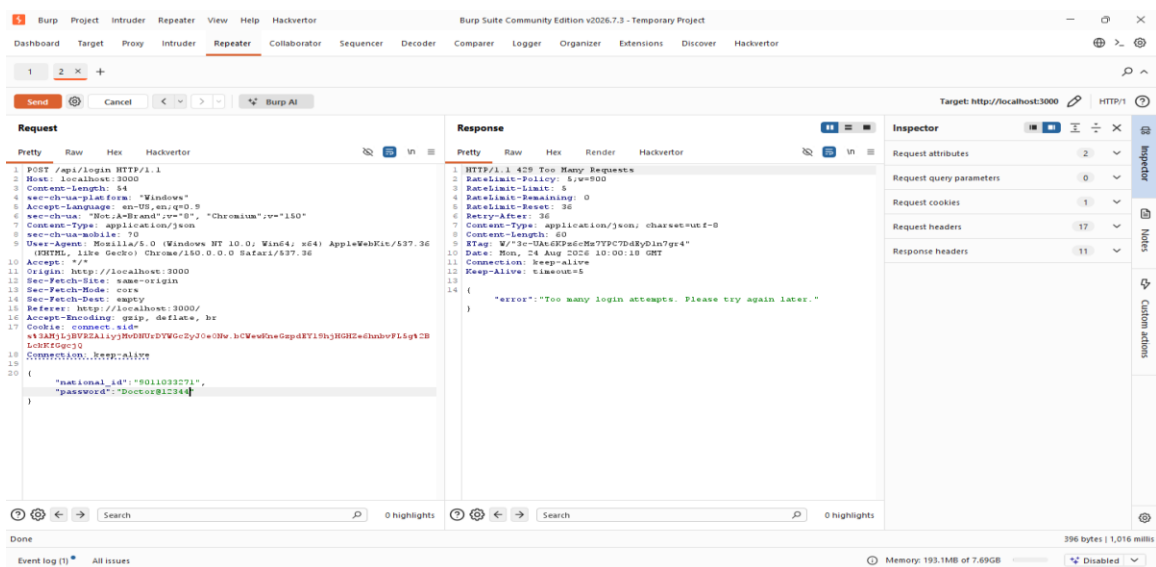


Figure 16 — After remediation: repeated login attempts are now throttled with 429 Too Many Requests.

7. Confirmed Impact Summary

The following outcomes were successfully achieved during the assessment, confirming real-world exploitability of the identified findings.

Evidence	Result
Credential material extracted	bcrypt password hash for the administrator account (id=1) retrieved via UNION-based SQL injection
Full database exposure	All 78 insurance case records returned from a single boolean-based SQLi request
Unauthorized medical record access	Patient file in Ismailia governorate (file 60) accessed by a Cairo-based doctor account with no authorization relationship
Client-side code execution	JavaScript alert() triggered automatically via a stored XSS payload in a medical note

8. Appendix A — Additional Testing Evidence (WSTG-CONF / WSTG-ATHN)

The following screenshots document supplementary configuration and authentication testing performed as part of the WSTG methodology. These confirm areas where the application behaved safely (negative findings), alongside the session-handling evidence referenced above.

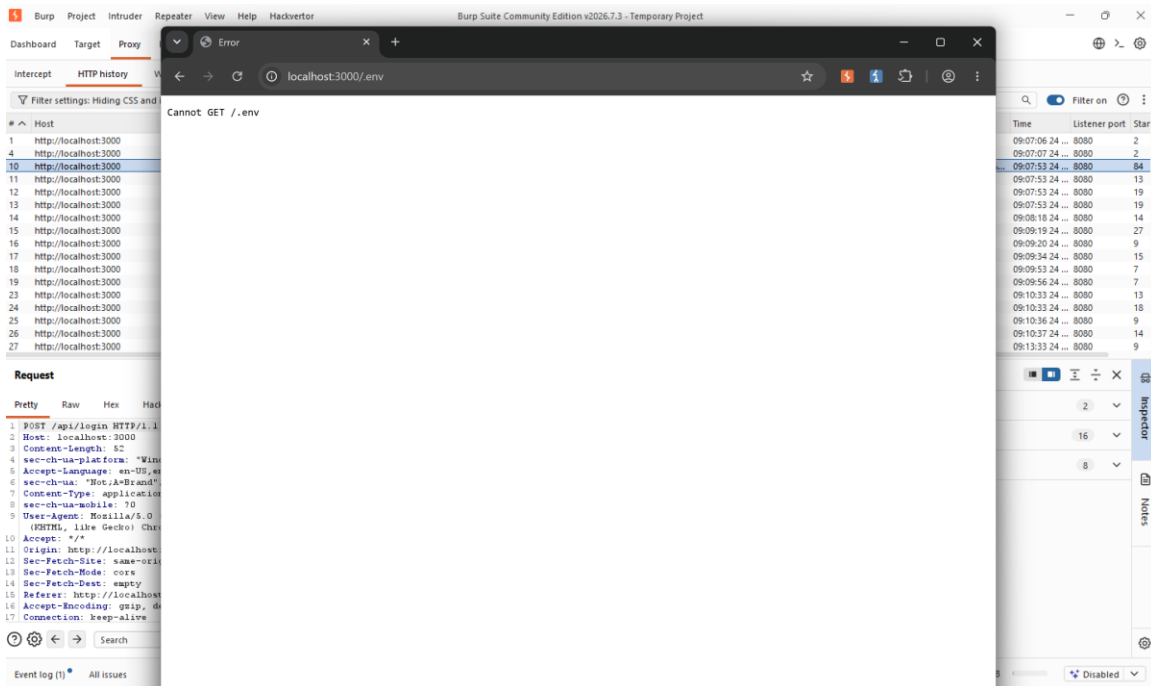


Figure 17 — No sensitive file exposure: /.env returns 404 (negative finding).

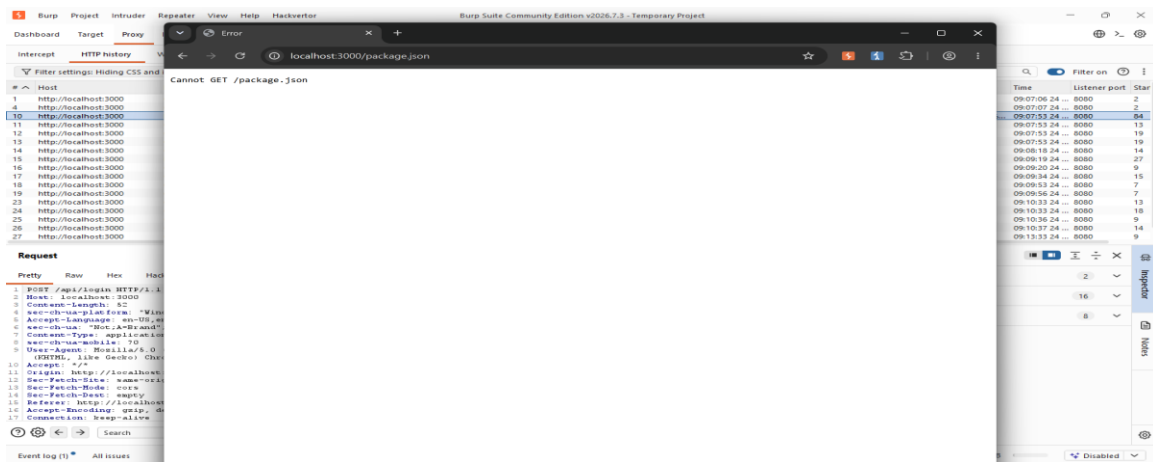


Figure 18 — No sensitive file exposure: /package.json returns 404 (negative finding).

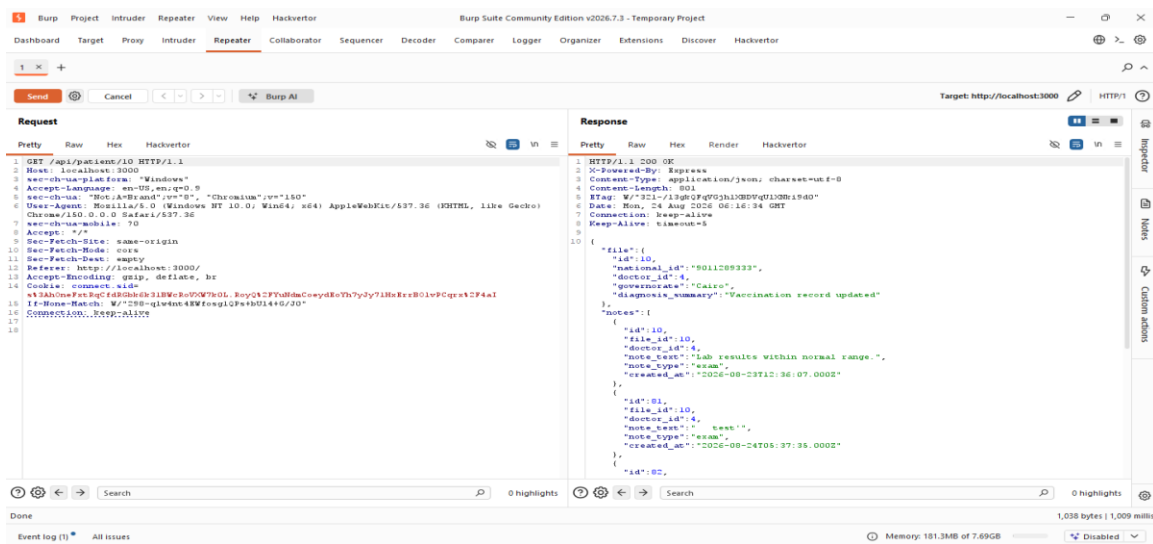


Figure 19 — OPTIONS request confirming only GET and HEAD are allowed on the patient endpoint, and disclosing the X-Powered-By header (Finding 6).

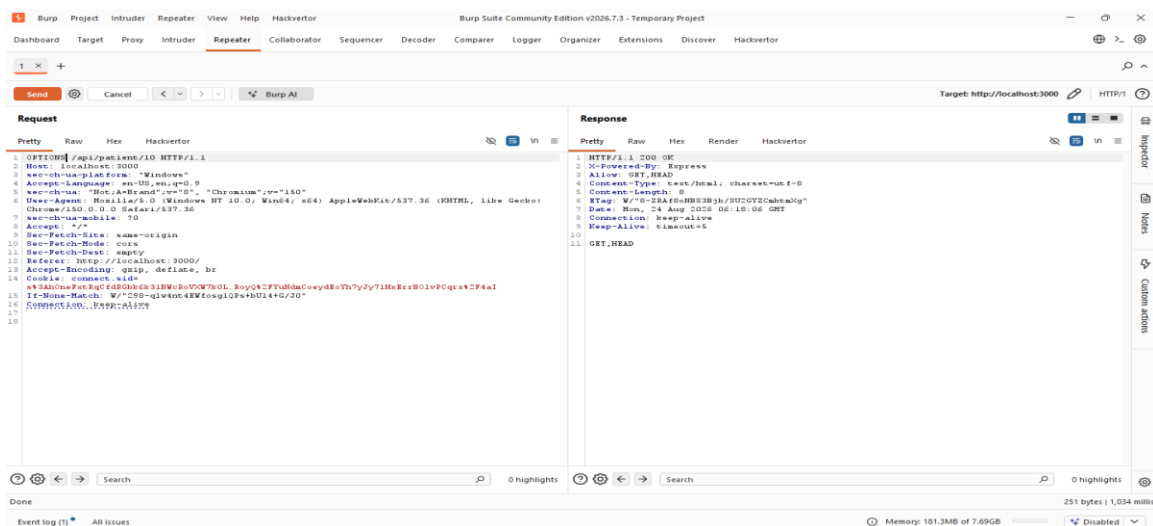


Figure 20 — DELETE method correctly rejected with 404 Not Found.

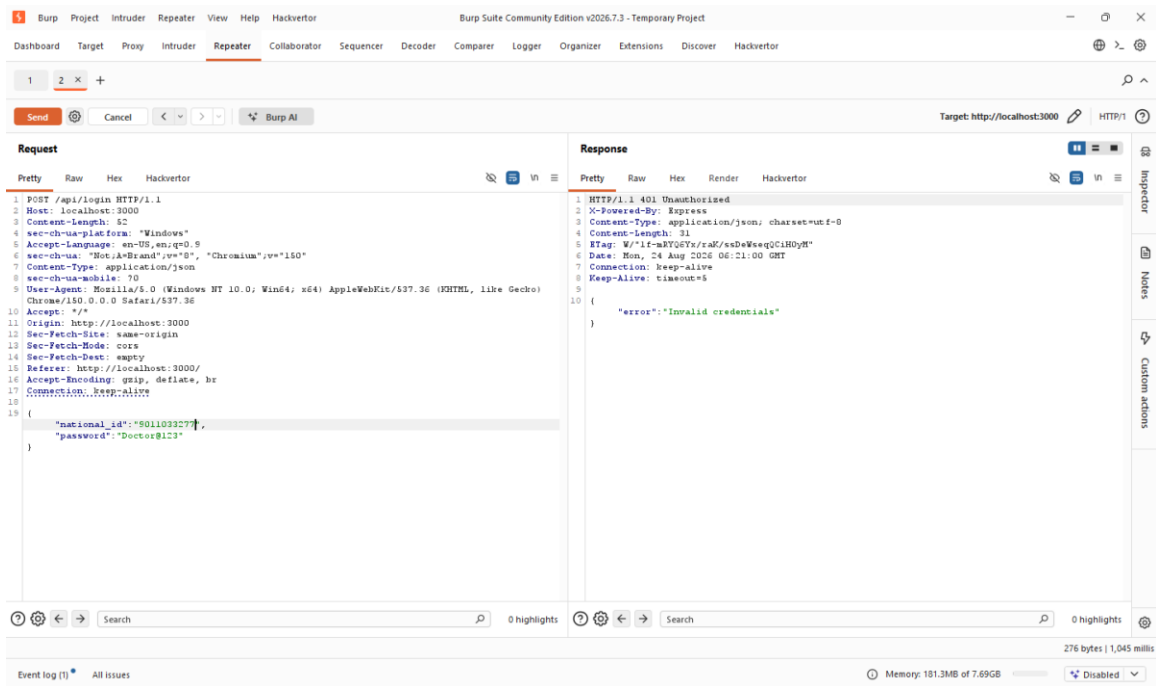


Figure 21 — Login with a non-existent national ID returns the same error message as an incorrect password, confirming no user-enumeration weakness.

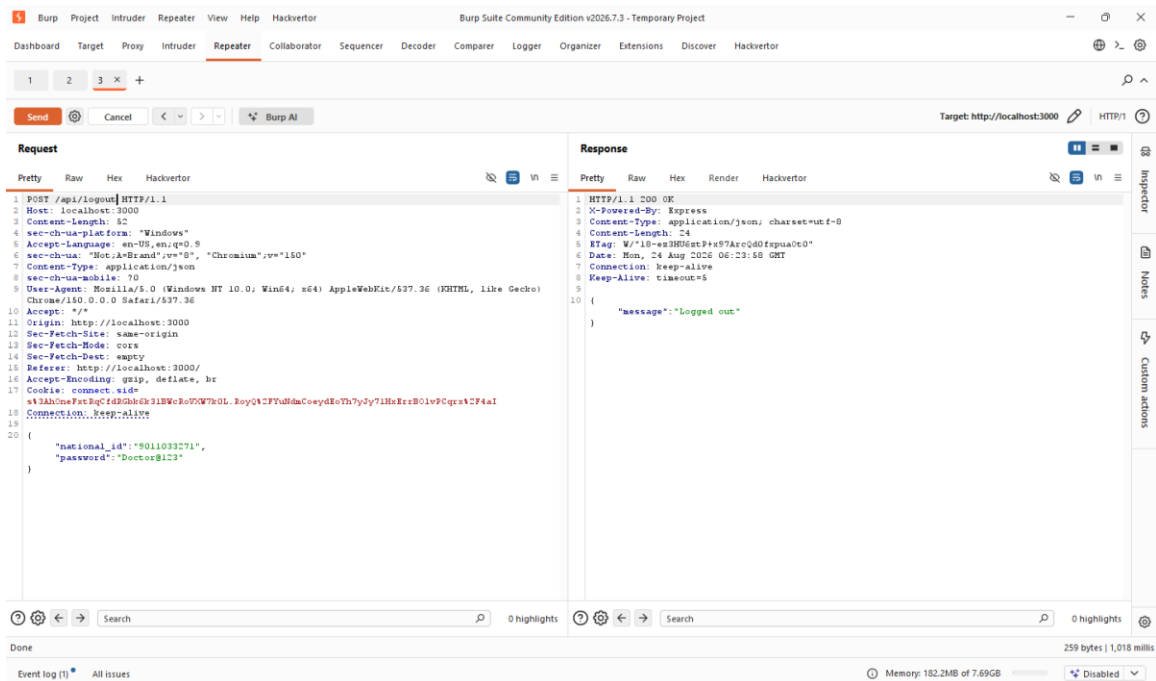


Figure 22 — Logout endpoint correctly invalidates the server-side session.

9. Appendix B — Remediation Verification Evidence

Additional before/after evidence supporting the remediation verification summarized in Step 6 of the Proof of Concept walkthrough.

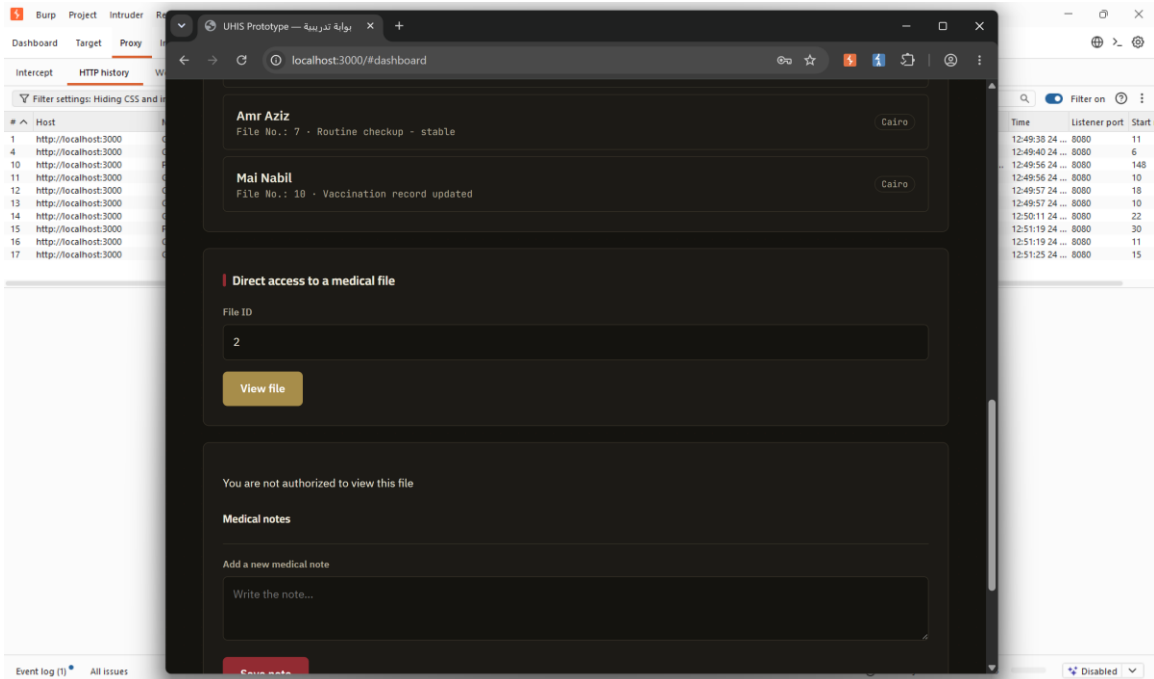


Figure 23 — Browser view confirming a 403 'not authorized' response for an out-of-scope patient file after remediation.

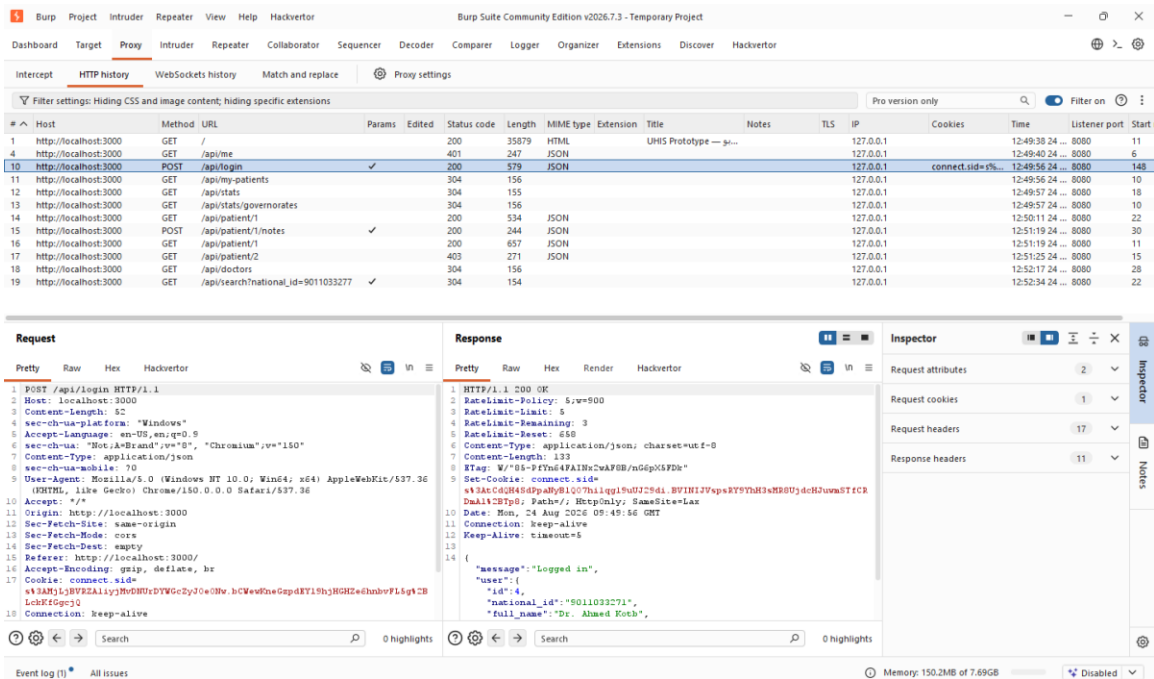


Figure 24 — Post-remediation login response showing both RateLimit-* headers and SameSite=Lax on the session cookie.

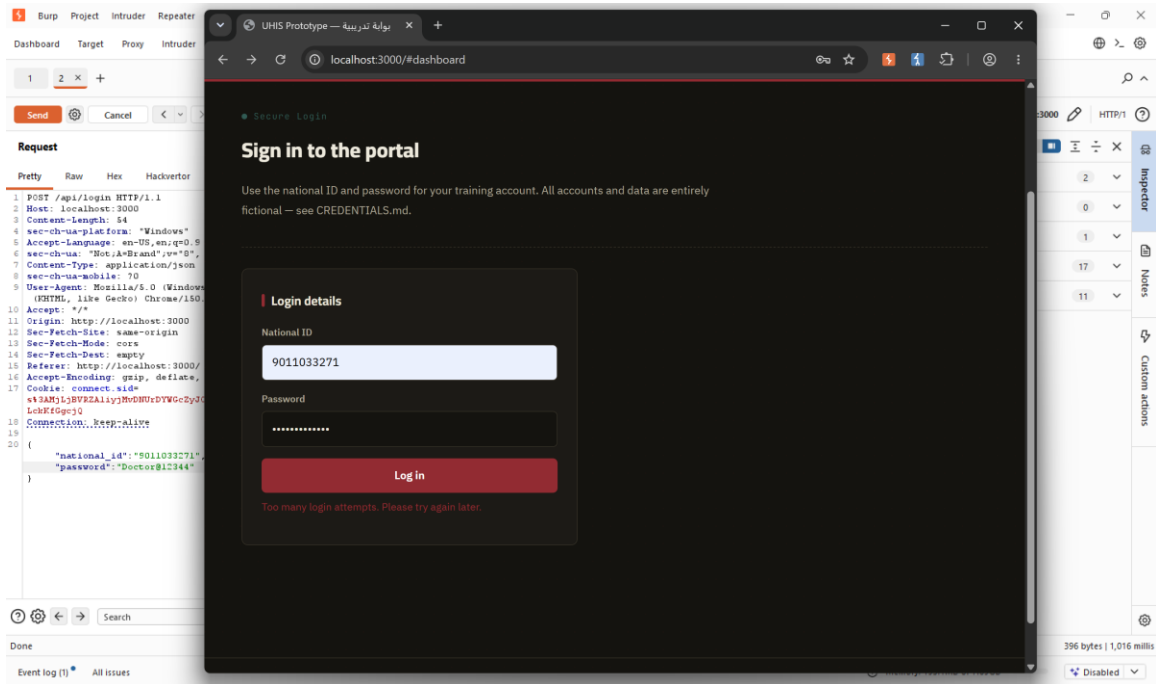


Figure 25 — The rate-limit block is now visible directly on the login form.

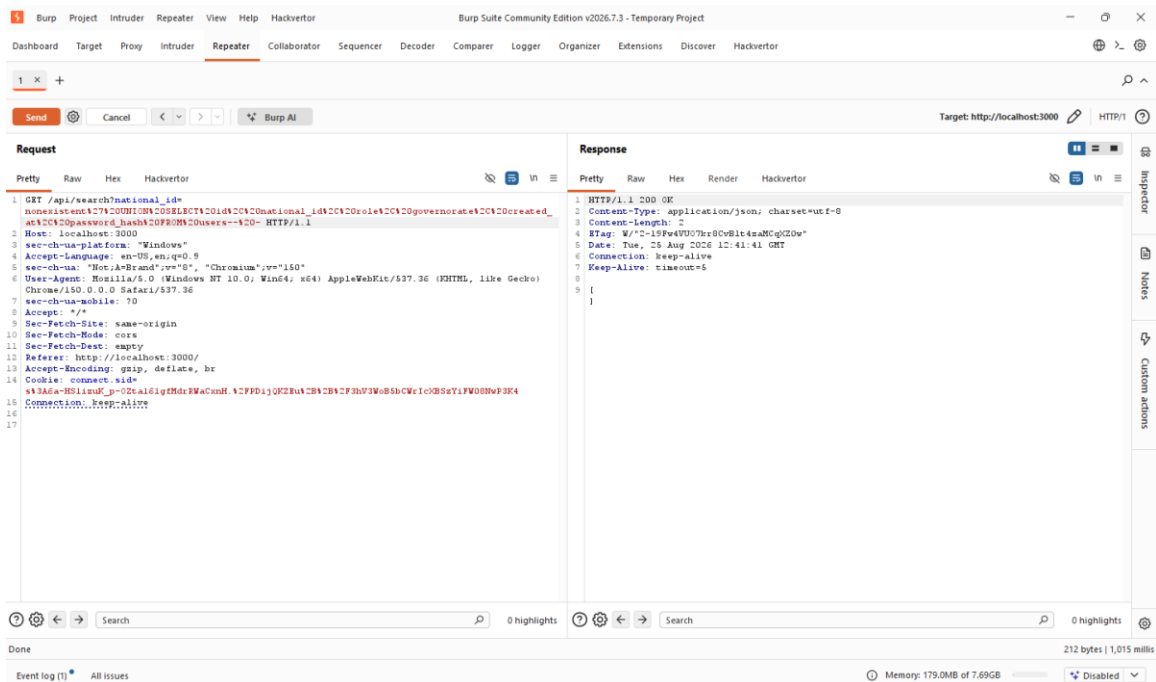


Figure 26 — The UNION-based SQLi payload now returns an empty, safe result after remediation.

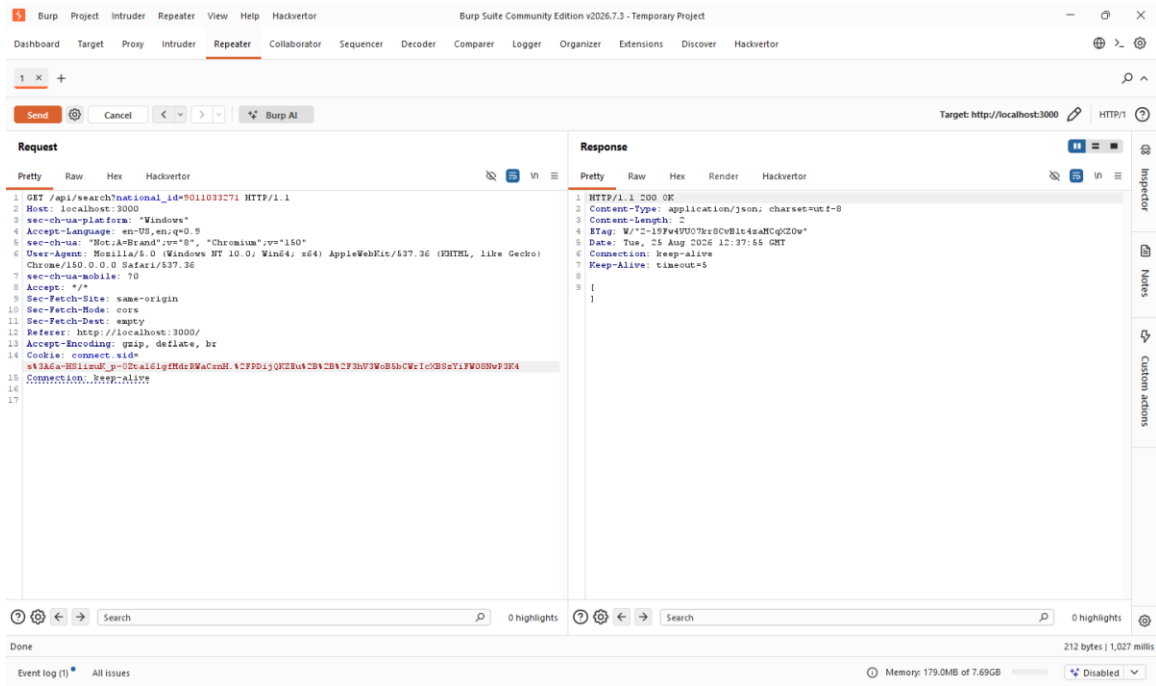


Figure 27 — Normal search functionality confirmed still working correctly after the SQL injection fix.

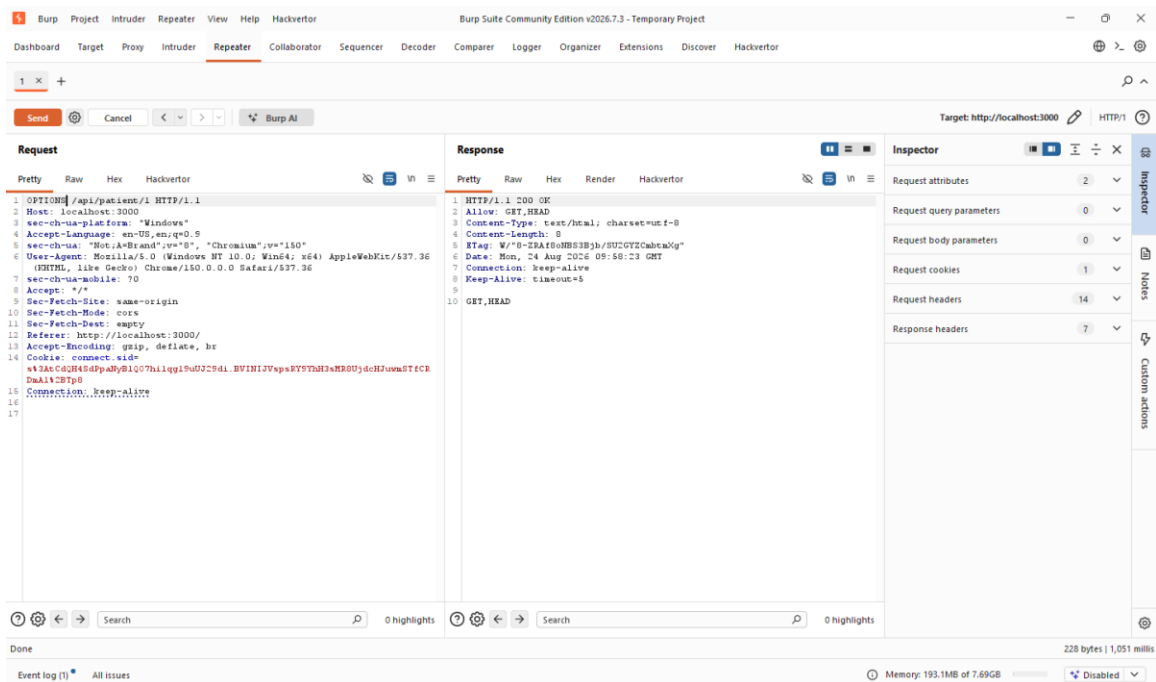


Figure 28 — Configuration behavior (OPTIONS method) unaffected by the remediation changes, confirming no regressions.

10. Recommendations Summary

The following prioritized remediation actions address the root causes identified in this assessment. All items below have already been implemented and verified in the current codebase.

- Replace all string-concatenated SQL queries with parameterized queries across the application (fixes Finding 1).
- Render all user-supplied text using `textContent` instead of `innerHTML`, or sanitize with an allow-list library (fixes Finding 2).
- Add server-side object-level authorization checks on every endpoint that accepts a resource identifier (fixes Finding 3).
- Apply per-IP/per-account rate limiting to the authentication endpoint (fixes Finding 4).
- Set the `SameSite` and `Secure` attributes on the session cookie (fixes Finding 5).
- Disable framework fingerprinting headers such as `X-Powered-By` (fixes Finding 6).
- Establish periodic access-control and input-validation review as part of routine secure development practice.

11. Conclusion

The penetration test against the Horus Byte UHIS Mini Portal successfully demonstrated how a small set of common web application weaknesses — unparameterized SQL queries, unsanitized HTML rendering, and missing object-level authorization — can compound into complete exposure of sensitive medical and credential data when tested using a structured, black-box methodology.

While individually some of these findings might be rated as low-to-medium risk, their combination — particularly the SQL Injection and Broken Access Control findings — represents a critical risk to patient confidentiality and system integrity. All six findings were remediated in the application's source code and independently re-verified using the same reproduction steps used during initial exploitation, confirming the fixes are effective without breaking legitimate functionality.

This engagement reinforced the value of following a structured testing methodology (OWASP WSTG) rather than testing only for a pre-known list of vulnerabilities — several supplementary findings (missing rate limiting, missing `SameSite`) were only discovered because the full methodology was followed end-to-end.

— End of Report —